

PROYECTO FINAL · FUNDAMENTOS DE ARQUITECTURA DE LLMS

Gateway LLM con seguridad de nivel producción

OWASP Top 10 for LLM Applications 2025, aplicado y demostrado en vivo

Alan Del Carpio Alosilla

Opción 4 · [fecha de presentación]

Presentación ejecutiva del proyecto final. Tres partes: por qué importa y qué buscamos; qué es OWASP Top 10 para LLMs y cómo cambia la seguridad; y la propuesta, con evidencia de que cada control funciona.

EL PROBLEMA

El gateway es el componente con mayor radio de impacto



Credenciales

Concentra las API keys de todos los proveedores de modelos.



Datos

Todo prompt y toda respuesta de la organización pasan por él.



Costo

Controla el gasto variable por tokens de cada aplicación.

Una falla aquí compromete confidencialidad, disponibilidad y presupuesto al mismo tiempo.

El gateway es la puerta única entre las aplicaciones y los proveedores de modelos. Por eso es el lugar donde una falla de seguridad tiene más impacto: credenciales, datos y costo se concentran en un solo componente.

Tres objetivos guían el diseño

01

Seguridad desde el diseño, no como parche

Aplicar de forma proactiva al menos tres mitigaciones del OWASP Top 10 para LLMs.

02

Controles demostrados, no solo documentados

Cada mitigación se prueba en vivo provocando el ataque que debe bloquear.

03

Auditoría útil que no se convierta en fuga

Registrar lo necesario para auditar, sin prompts ni credenciales.

Los tres objetivos de aprendizaje del proyecto, convertidos en objetivos de diseño verificables. El segundo es el diferenciador: una mitigación de papel existe en un documento; una mitigación real se demuestra provocando el ataque.

REQUERIMIENTOS

Lo que el sistema debe cumplir

Funcionales

- Un único endpoint por el que pasa toda llamada a un LLM
- Rate limiting verificable · LLM10
- Sanitización de la entrada antes del prompt · LLM01
- Ninguna API key en código ni en logs · LLM02
- Una cuarta mitigación elegida · LLM07

No funcionales

- Logging con metadatos de auditoría, sin prompt ni keys
- Degradación controlada ante fallos del proveedor, sin exponer trazas internas

Criterio de aceptación: cada control tiene evidencia reproducible **antes y después** de activarlo.

El enunciado exige un punto único de entrada, tres mitigaciones obligatorias (LLM10, LLM01, LLM02) y una cuarta a elección; elegimos LLM07. Los requisitos no funcionales son logging seguro y degradación controlada. El criterio que atraviesa todo: evidencia antes/después.

PARTE 2 · EL MARCO DE REFERENCIA

OWASP Top 10 for LLM Applications es el estándar de la industria para auditar sistemas con modelos de lenguaje

Diez categorías de riesgo propias de las aplicaciones con LLM, publicadas por el OWASP GenAI Security Project en su edición 2025. Complementan, no reemplazan, la seguridad web tradicional.

Gateway LLM seguro · 05

OWASP es la referencia que usan equipos de seguridad y auditores. Su lista para LLMs describe riesgos que no existen en el software tradicional, o que cambian de forma: por eso un gateway no puede protegerse solo con controles web clásicos.

Diez riesgos; el gateway cubre los cuatro resaltados

ID	Categoría	En una frase
LLM01	Prompt Injection	Entradas que alteran el comportamiento del modelo
LLM02	Sensitive Information Disclosure	Fuga de credenciales, datos personales o internos
LLM03	Supply Chain	Modelos, datos o librerías de terceros comprometidos
LLM04	Data and Model Poisoning	Datos de entrenamiento o ajuste manipulados
LLM05	Improper Output Handling	Usar la salida del modelo sin validarla
LLM06	Excessive Agency	Más permisos o autonomía de los necesarios
LLM07	System Prompt Leakage	Exposición de las instrucciones internas del sistema
LLM08	Vector and Embedding Weaknesses	Debilidades en RAG y bases vectoriales
LLM09	Misinformation	Respuestas falsas presentadas como ciertas
LLM10	Unbounded Consumption	Consumo sin límites: costo y denegación de servicio

Los nombres de las categorías se mantienen en inglés porque así aparecen en el estándar y en las auditorías. Las cuatro resaltadas son las que se materializan en el propio gateway. LLM03, LLM04, LLM06 y LLM08 quedan fuera porque el gateway no entrena modelos, no ejecuta herramientas ni recupera documentos en esta versión.

POR QUÉ HACE FALTA UN MARCO PROPIO

Un LLM cambia las reglas de la seguridad

Dimensión	Software tradicional	Aplicación con LLM
Instrucciones y datos	Separados: el código manda, la entrada es dato	Mezclados en el mismo texto: la entrada puede dar órdenes
Comportamiento	Determinista y verificable caso a caso	Probabilístico: la misma pregunta puede variar
Costo	Fijo, ligado a la infraestructura	Variable por token: el abuso se convierte en factura
Activos nuevos	Código, datos y credenciales	Además: system prompt, contexto y la propia salida del modelo

Consecuencia: la salida del modelo es **dato no confiable** y cada control debe asumir que otro puede fallar.

La diferencia de fondo: en un LLM, instrucciones y datos viajan en el mismo canal de texto. Por eso existe la inyección de prompts, y por eso la defensa debe ser en capas. El costo por token convierte el abuso en un problema financiero, no solo técnico.

SELECCIÓN

Cuatro riesgos que se materializan en el gateway

LLM10 · Unbounded Consumption

Aquí se concentra el gasto: una sola key comprometida puede agotar presupuesto y capacidad.

LLM01 · Prompt Injection

Aquí la entrada no confiable se convierte en prompt; controlarlo una vez protege a todas las apps.

LLM02 · Sensitive Information Disclosure

Custodia las credenciales y ve todo el tráfico: sus logs son un objetivo de alto valor.

LLM07 · System Prompt Leakage

El system prompt vive en el gateway; su fuga revela reglas de negocio y facilita otros ataques.

LLM05 queda como quinta mitigación: depende en parte de cómo cada cliente usa la salida.

Criterio de selección: que el riesgo ocurra en el propio gateway, no en una capa que no controla. LLM07 se prefirió sobre LLM05 como cuarta porque su mitigación es enteramente responsabilidad del gateway y su evidencia es binaria.

Ocho controles en serie protegen cada solicitud

Un único endpoint, **POST /v1/chat**. Si un control falla, la solicitud se bloquea.



El adaptador habla con el proveedor (Anthropic, OpenAI, Ollama o un mock determinista); su respuesta vuelve como dato no confiable.

Recorrer el pipeline de izquierda a derecha. Detenerse en el 6: el adaptador es el único módulo que conoce la credencial del proveedor. La respuesta del modelo vuelve como dato no confiable y debe pasar por la guardia de salida antes de llegar al cliente. En modo presentación cada control aparece al avanzar.

Cuatro decisiones hacen la seguridad demostrable



Fail-closed

Si un control falla, la solicitud se bloquea; nunca se deja pasar.



Dos perfiles

El mismo código corre en **baseline** (reproduce el ataque) y **secure** (lo bloquea).



Mock determinista

Un proveedor simulado da evidencia reproducible; un modelo real no siempre responde igual.



Logging por allowlist

Solo se escriben campos declarados; un dato sensible no puede llegar al log por error.

Estas cuatro decisiones son las que permiten afirmar que la seguridad es real y no de papel. La más importante para la evaluación es la de los dos perfiles: el mismo binario muestra el ataque funcionando y luego bloqueado.

MITIGACIÓN · LLM10 UNBOUNDED CONSUMPTION

La cuota por cliente detiene el consumo sin límites

Cuota por identidad de la key, no por IP · tope de tamaño y de tokens por solicitud.

SIN PROTECCIÓN · baseline

15 / 15

Solicitudes en ráfaga aceptadas: las 15 llegan al proveedor y se facturan.

CON PROTECCIÓN · secure

10 + 5

10 aceptadas y 5 rechazadas con 429, aun rotando la IP. Otro cliente conserva su cuota.

Gateway LLM seguro · 11 · Evidencia: make attack-llm10 · tests/security/test_llm10_rate_limit.py

Ataque: 15 solicitudes en ráfaga con la misma key, cambiando la IP en cada una. Sin protección pasan todas. Con protección, el límite de 10 por minuto se aplica a la identidad del cliente, así que cambiar la IP no sirve. Las respuestas 429 indican cuándo reintentar. Cifras de la ejecución real de la suite.

MITIGACIÓN · LLM01 PROMPT INJECTION

La inyección se detiene antes de llegar al modelo

Aislamiento de roles · normalización Unicode · reglas de detección · delimitación de la entrada.

SIN PROTECCIÓN · baseline

PWNED

El modelo obedece una orden oculta, disimulada con un carácter invisible.

CON PROTECCIÓN · secure

400

Rechazada antes de salir del gateway: 0 llamadas al proveedor y 0 tokens gastados.

Gateway LLM seguro · 12 · Evidencia: make attack-llm01 - tests/security/test_llm01_prompt_injection.py

Ataque: una traducción inocente que esconde la orden "ignora las instrucciones anteriores", con un carácter invisible dentro de la palabra para burlar filtros simples. Sin protección, el modelo responde PWNED. Con protección, el gateway normaliza el texto, detecta la regla y rechaza la solicitud. Tampoco se acepta que un cliente envíe su propio mensaje de sistema. Advertencia honesta: la detección heurística es evadible; por eso es una capa entre varias.

MITIGACIÓN · LLM02 SENSITIVE INFORMATION DISCLOSURE

Ninguna credencial en código, logs ni errores

Secretos fuera del código · keys de cliente como hash · logging por allowlist · errores sin trazas.

SIN PROTECCIÓN · baseline

4

Secretos hallados por gitleaks en el log, además del dato personal del usuario y la traza interna expuesta al cliente.

CON PROTECCIÓN · secure

0

Secretos en logs y en todo el historial git. El cliente recibe un error 502 limpio con un identificador de seguimiento.

Gateway LLM seguro · 13 · Evidencia: make attack-llm02 · make scan · tests/security/test_llm02_sensitive_info.py

Ataque: provocar un fallo de credencial en el proveedor con un prompt que incluye un DNI. Sin protección, el cliente ve la traza interna y el log guarda la credencial, la key del cliente y el DNI: gitleaks encuentra 4 secretos. Con protección, error 502 limpio y 0 hallazgos, incluso revisando todo el historial de git. El operador sí recibe una alerta. Las credenciales de la demo son ficticias.

MITIGACIÓN · LLM07 SYSTEM PROMPT LEAKAGE

Las instrucciones internas no salen del gateway

Marcador secreto (canary) en las instrucciones · detección de coincidencias en cada respuesta.

SIN PROTECCIÓN · baseline

Fuga

Una petición disfrazada de tarea de documentación obtiene el system prompt completo, con su canary.

CON PROTECCIÓN · secure

0 fugas

Respuesta neutra, aun con la sanitización de entrada apagada a propósito: la salida tiene su propia defensa.

Gateway LLM seguro · 14 · Evidencia: make attack-llm07 · tests/security/test_llm07_system_prompt_leak.py

Ataque: pedir al modelo que transcriba su configuración inicial, como si fuera para documentar el sistema. Sin protección, la respuesta contiene el system prompt y el marcador secreto. Con protección, el modelo sí filtra, pero el gateway lo detecta en la salida y devuelve una respuesta neutra. En esta prueba apagamos la sanitización de entrada para demostrar que la defensa de salida funciona por sí sola: defensa en profundidad.

Auditoría completa sin convertir el log en una fuga

Se registra

- Fecha, endpoint y resultado
- Cliente seudonimizado
- Regla de seguridad que actuó
- Latencias del gateway y del proveedor
- Tokens consumidos

No se registra

- Contenido del prompt y de la respuesta
- API keys y cabeceras de autorización
- System prompt y su canary
- El fragmento del ataque detectado
- Trazas internas hacia el cliente

Garantía estructural: el evento de auditoría solo acepta campos declarados; un dato sensible no puede escribirse por error.

Pregunta 3 del video: qué decidimos no registrar y por qué. Un log suele tener controles de acceso más débiles que la aplicación; si guardara prompts o credenciales, se convertiría en el activo más sensible del sistema. Registramos lo necesario para auditar y operar: quién, cuándo, qué resultado, qué regla, cuánto tardó y cuánto consumió.

Ante fallos del proveedor, errores claros y sin trazas

Falla del proveedor	Sin protección	Con protección
Tiempo agotado	El cliente queda esperando	504 dentro del límite configurado
Error interno	500 con traza interna	502 tipado, sin detalles internos
Proveedor saturado	500 con traza interna	503 tras reintentos acotados
Fallos repetidos	Se sigue insistiendo	Circuito abierto: 503 inmediato

Cada error incluye un identificador para que soporte lo encuentre en el log de auditoría.

Requisito no funcional del enunciado. Sin protección, tres fallos distintos producen el mismo error 500 con la traza interna, y un timeout deja al cliente colgado. Con protección, cada falla tiene un código y un mensaje claros. Tras tres fallos seguidos, el circuit breaker deja de enviar solicitudes al proveedor caído.

Cada criterio de la rúbrica tiene evidencia

Criterio	Peso	Cómo se cumple
Funcionalidad y código	30	Endpoint único; un módulo por control
Seguridad y arquitectura	20	Secretos fuera del código, logging seguro, degradación controlada
Mapeo OWASP y demos	25	Cuatro controles con evidencia antes y después
Documentación	10	Mapeo OWASP que un tercero puede reproducir
Video	15	Las cuatro demos antes/después, no solo la mínima

51 pruebas automatizadas en verde

0 secretos en código, historial y logs

Trazabilidad directa con los 100 puntos de la rúbrica. Las cifras de abajo son de la ejecución real: la suite completa pasa y gitleaks no encuentra secretos ni en el código, ni en el historial git, ni en los logs del perfil seguro.

Lo que el diseño no resuelve, y cómo se compensa

Limitación	Tratamiento
La detección de inyección es evadible con paráfrasis	Es una capa entre cuatro; la guardia de salida es la segunda línea
Falsos positivos que bloquean usos legítimos	Corpus legítimo en las pruebas; reglas desactivables una a una
Un modelo real responde de forma no determinista	Evidencia oficial con el mock; el modelo real, como demo complementaria
Cuota en memoria con varias instancias	Redis como almacén compartido de contadores

Declarar las limitaciones es parte de la seguridad. Ningún filtro de inyección es perfecto; por eso el diseño no depende de uno solo. Para una audiencia ejecutiva: el riesgo residual existe, está identificado y tiene tratamiento.

PRÓXIMOS PASOS

Con más tiempo, la quinta mitigación sería LLM05

01 • LLM05

Manejo de la salida

Entregar siempre la salida como dato y marcar código o HTML antes de que un cliente lo ejecute.

02

Clasificador

Un modelo especializado como segunda opinión frente a las reglas de detección.

03

Key Vault

Secretos gestionados en Azure Key Vault para el despliegue productivo.

04

Streaming

Guardia de salida por ventanas para respuestas que llegan por partes.

Pregunta 4 del video. LLM05 cierra el ciclo: LLM01 protege lo que entra al modelo y LLM05 protege a quienes consumen lo que el modelo produce, incluso si una inyección no detectada logra manipular la salida.

EN SÍNTESIS

Una mitigación real se demuestra provocando el ataque que debe detener

4

categorías OWASP cubiertas

51

pruebas automatizadas

0

secretos en código y logs

Repositorio: [\[enlace al repositorio\]](#) · Mapeo completo en docs/OWASP_MAPPING.md

Cierre: el mensaje para la audiencia es que la seguridad del gateway no está en un documento, está demostrada. Invitar a preguntas y, si hay tiempo, ejecutar una demo en vivo con `make attack-llm01` en ambos perfiles.